Centro de Informática
UFPE

# Implementing Product Line Variabilities

Enzo Gurgel Bissoli(egb2)

UFPE CIn

2026-03-23

# What is a product line

# Abstract

- How to deal with genericity at code level

# Introduction

- How to combine parts is very challenging

# How to compose a product line

Centro de Informática UFPE

- Generic assets (Fill the requirements for multiple products in a product line)
- Specifc assets (Fill the requirements for a single product in a product line)
- decision model (structure variability)

"A product is a sesqunece of instances of decisions models"

**variability and composition rules, must be addressed at the code level**

# Related Work

*James Coplien* -> “realization of commonalities and variabilities”

*David Sharp* -> “anticipated variabilities between distinct software versions, parameters” : a parameter can be analyzed over (Scope, flexibility, efficiency and applicability)

-Mikael Svanhberg“ -> Taxonomy of solutions (TODO: might be worth a look!)

*Mira Mezini* -> Variational Object Oriented Programming(VOP) : distinct kinds of variations, extension to oop that models context behavior

*Krzystof Czarnecki* -> Generative Programming : Moddilng software "system famielies" s.t particular requiriments -> optimized end product on demand (templates filling?) :: Aspect-oriented programming, Subject oriented programming and VOP

# Requirements for Implementation

Centro de Informática UFPE

"Every variability encountered in a product line context, can be connected with corresponding feature"

- How to deal with feature scattering, tangling in source files and modules?

. Feature Type: (Mandatory, Optional, Alternative, Mutually Inclusive, Mutually Exclusive) . Variability Type: (Positive, Negative, Optional, Alternative, Function, Platform/Environment) . Binding Times: (Compile-time, Link-time, Runtime, Update-time/Post-runtime) Need to find variation points and location of variability

“Management Traceability?” -> Matrices that connect artifacts -> Hard to scale, when a new product is introduced need to re-do variability analysis : tries to solve by stepwise-implementation, but a hollistic view is required

# Framework For Comparison

Centro de Informática UFPE

- Aggregation/Delegation approach

-> Allow objects to forward requests to a delegator, so that object can have minimum implementation required : Requires references to another object (bloat)

: Requires to define a sequence of operations (design intensive)

:: Variability is handled by allowing the delegated object to provide the variation point

(+) Work with optional features

(-) Alternative features break

(-) Can’t scale

(!) Compile time approach, but can be Link-time if shared libs are being used

- Inheritance

-> Extensions to Subclasses and commonalities to superclasses : Subclass can overwrite attributes or methods

: Virtual inheritance, allows class composition during compiletime

:Multiple inherintance -> generates the diamond problem which superclass comes first in the inheritance tree

:Mixin inheritance -> decorator pattern

: Prototype pattern -> a class has a metaclass that inherits from and behavior can be hooked during inheritance tree

(-): growth of distinct variabilities implies in more and more subclasses (complexity explosion)

(-): hard to deal with options due to name clashing

- Parametrization

-> Reusable software as lib of parametrized components

(+) Avoid code replication, by enforcing design around set of variables

(+) increase reusability and traceability

(-) very complex to do the required analysis

(-) depedent on parametrization feature of programming languages (maybe proof oriented programming languages are better for this such as Idris)

(++) every possible binding time for variability can be deal with when considering parametrization heavy design

- Dynamic Class Loading

-> All classes are loaded into memory as soon as needed (possibility of standardizing class lifecycle)

(+) Allow the Product to query it's own state (lisp?(Adaptative software product lines)) (+) traceability is easy

- Static Libs

-> change libs change implementation, can be selected during compile time

- Dynamic Link Libs

-> On-demand libs provide variability during runtime

(+) provide parametric components

(?) Stuff like CORBA and Java Beans provide similar functionality

- Conditional Compilation

-> Control blocks of code to be enabled/disabled during pre-compile-time (Directives)

(+) Encapsulation of multiple implementations in single modules, reducing scattering

(+) Separation of variabilities (Includes in C for instance)

(-) Lack recursion support or control flow support

(-) Need central include like a “index file” to toggle implementation

- Frames

-> *Paul Basset's* adaptable entities, provide hierarchical reuse, provide a src file that a parent module can modify as it wishes(insertion or replacement of code, need to set frame parameters), a pre-compile-time approach

(-) Hard to have an overview as the code evolves

- Reflection

-> Hability of program to manipulate it's own data during runtime, a metaprogramming approach

(+) Forces conceptual integrity by classifying specific objects in generic objects thus creating protocols of communications between distinct entities in a layer, but equal in another

(+) Can be combined with Dynamic Class loading to enforce adaptability

(-) Hard to debug, understand and mantain

(-) Optimizations are possible during post-runtime

- Aspect Oriented Programming (AOP)

-> Developed by *Xerox Parc*, modularization of non-essential business features that every module should have, features such as sandboxing, logging, authorization etc

(+) Provide composition of “cross cutting concerns” to customize objects in Join Points

(?) Variations such as Subject oriented programming exist

(+) Interfaces can be provided by “aspects” to enhance objects

(+) Easy to trace, because of inheritance tree

(-) Hard to provide “aspects” for multiple options (variabilites)

(?) They seem to not be hierarchical based (maybe composition?)

(?) Binding time is during compilation

(+) Active libs exist to provide runtime Aspect injection

# Design Patterns

-> Can provide solution to variability, because they recognize it and create specific solutions to manage it

(-) hard to manage traceability, because code can be mutated

(-) Objected oriented + parametrization heavy (so requires lot of design though)

# Examples and analysis

Centro de Informática UFPE

```
public void A2() {
  delegationObject.A2();
}

public void A3(){
  foo1();
  foo2();
  delegationObject1.foo3();
  foo4();
  delegationObject2.foo5();
  foo6();
}
```

-> “A2 is only implemented in the premium products” they do this to provide interface uniformity and avoid feature flagging

(>) A exception is raised on lower tier products

(D1) Option “Functionaly F2 is supported” {yes/no} (D2) Alternative “Functionality F3” {std,extended,pro}

(*) They could just do nothing and signal in the methodcall in a more verbose way

(*) They could also use the decorator pattern to signal intent, but you would need to know the meaning of delegate to easily understand it

```
#ifdef F2
/*include code*/
#endif

#ifdef F3Extended
#endif

#ifdef F3Pro
#else
#endif
#endif
```

(*) Can't do this in SaaS or any cloud based product, runtime adaptation is a must

(*) Could do a dynamic index file override approach such that features are toggled during runtime in a centralized place

- The Builder Pattern

### 5.3 Design Patterns - Dynamic Link Libraries

The Builder Pattern is an example of a creational pattern that can be used for loading variant code at runtime. This pattern is suitable when the logic for constructing complex objects must be separated from the users of those objects and when this logic must facilitate the building of variations [18]. The structure of the pattern is depicted in Figure 6

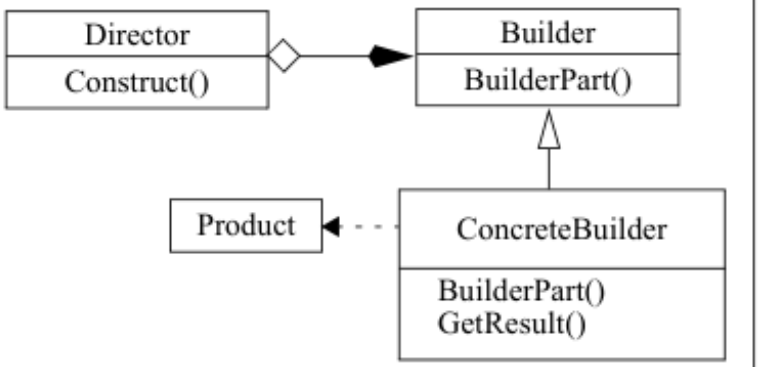


**Figure 6. The Builder Pattern**

using UML notation.

(>) Treating DLL's as builders, so the director can invoke each during variation points

# (*) Good for composing UI components